

GridLab

Local LAN Zero-Trust Peer Compute & GPU Broker

Open-Source University Infrastructure

The Problem — Campus Compute Bottlenecks

Free-Tier Cloud Limits

- Google Colab GPU sessions time out mid-training.
- Strict memory and runtime caps on free tiers.
- Long queue times on shared university servers delay coursework and research.

High Hardware Costs

- Students limited by consumer laptop VRAM and TDP.
- Complex CAD, circuit simulation, and ML workloads exceed local resources.
- Budget constraints prevent timely hardware upgrades for labs.

The Wasted Asset — Idle Local Hardware

University Computer Labs

Hundreds of high-end GPUs sit fully idle overnight and during off-peak hours.

Student Hostels

High-performance gaming laptops remain plugged in and unused while owners sleep.



Terabytes of VRAM and thousands of CUDA cores sit unutilized across campus subnets every single night.

The Solution — GridLab Overview

Open-source, peer-to-peer compute-sharing daemon designed for campus LANs. Enables local GPU sharing without external cloud dependency while preserving host control.

Zero Cloud Dependency

All traffic remains on the internal LAN — no external data egress.

Zero-Trust Security

Ephemeral, isolated sandboxes with strict resource and access controls.

Non-Intrusive

Host priority enforced; guest jobs yield immediately on owner activity.

Pillar 1 — Zero-Trust Sandboxed Execution

Trigger

Daemon activates only when host CPU/GPU usage drops below 10%.

Containerization

Launches lightweight, ephemeral Docker or WebAssembly (Wasm) containers for guest tasks.

Isolation

Guest tasks cannot access host filesystems, environment variables, or local memory.

Ephemeral Lifetime

Sandbox dissolves on completion or immediate interruption; no persistent guest state on host.

Pillar 2 — Local LAN Auto-Discovery

- **Protocol**

Uses mDNS / DNS-SD for peer auto-discovery across campus subnets.

- **Zero Internet Overhead**

Bypasses ISP bandwidth caps and university perimeter firewalls for compute traffic.

- **High Bandwidth & Low Latency**

Leverages gigabit campus networks for fast model weight transfer and distributed dataset streaming.

Pillar 3 — Host Resource Guardrails



Input Detection

Pauses guest jobs within milliseconds when host input (mouse/keyboard) is detected.



Thermal & Power Limits

Automatic throttling or stop when temperatures exceed safe thresholds or battery falls below configured cutoff.



Idle State Management

Interrupted tasks migrate or checkpoint to another peer with minimal progress loss.

System Architecture Workflow

Discovery

Find peers via mDNS on the campus network.

Negotiation

Agree on container job and RAM/VRAM needs.

Execution

Run when host idle (<10%) using Docker/Wasm.

Handoff

Checkpoint and migrate over LAN for completion.

Security & Trust Model

Network Isolation

Traffic confined to campus LAN; no exposed external ports by default.

Encrypted Peer Transport

Mutual TLS authentication for all peer-to-peer communications.

Zero File Access

Container policies enforce strict read/write limitations; host drives are not mounted.

Resource Caps

Cgroups and scheduler limits prevent GPU/CPU overload and protect host longevity.

Summary & Open-Source Vision

Key Takeaways

- Unlocks hidden campus GPU capacity without expensive hardware purchases.
- Provides students low-latency, zero-cost access to heavy compute on LAN.
- Designed and governed by the campus community for internal networks.

Next Steps

- Publish repository and technical roadmap (link placeholder).
- Alpha testing on selected campus subnets.
- Deploy node daemons across lab systems; collect telemetry and safety metrics.